

Arab Academy for Science , Technology & Maritime Transport
College of Computing and Information Technology



Data Structures and Algorithms (CS212)

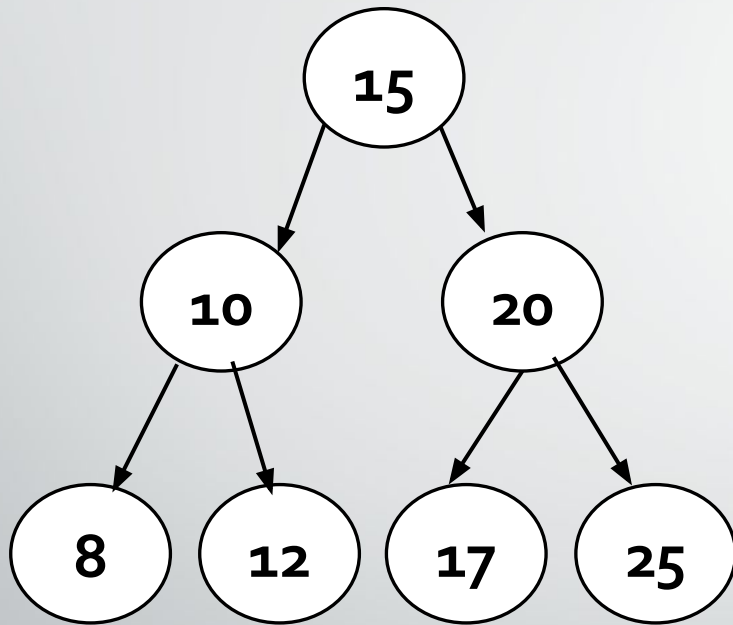
Section 07 : Tree Traversals

Eng. Youssef A. Mehanna



Tree Traversals

- ✓ Traverse a Tree means visiting all nodes in the tree

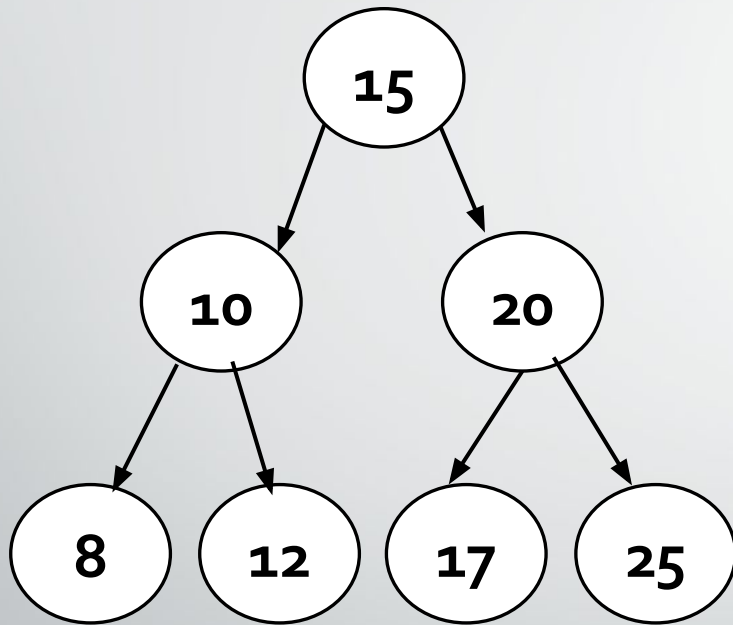


- ✓ **Breadth-First
(Level-Order)**

- ✓ **Depth-First :**
Preorder
Inorder
Postorder

Tree Traversals

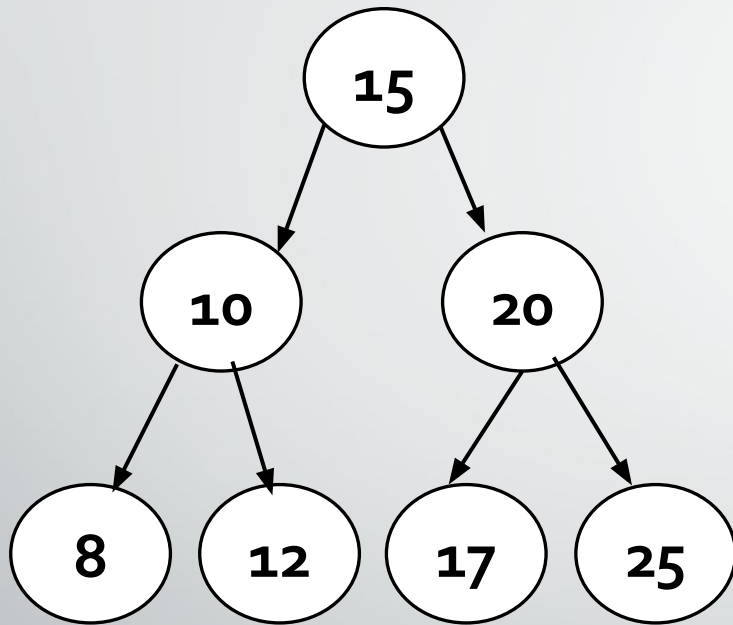
✓ **Breadth-First: visit level by level**



15 10 20 8 12 17 25

Tree Traversals

✓ Depth-First



1- Preorder (DLR)

`<root><left><right>`

2- Inorder (LDR)

`<left><root><right>`

3- Postorder (LRD)

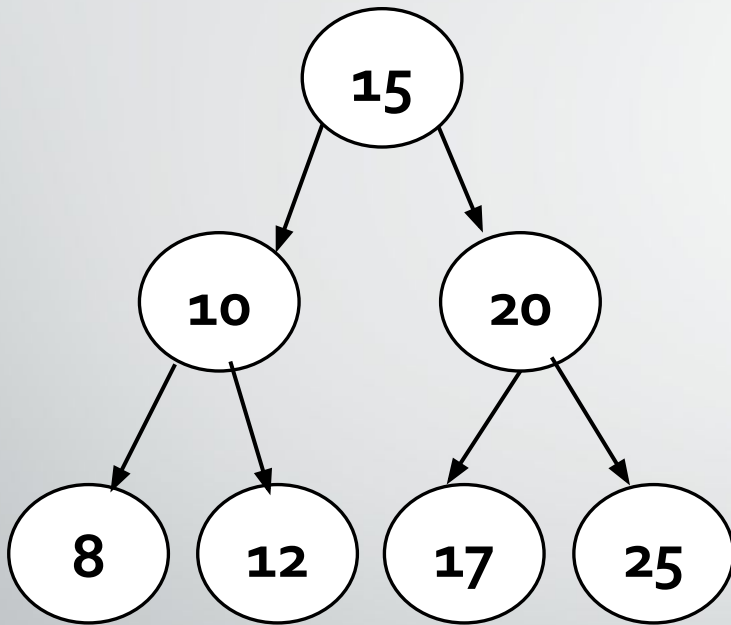
`<left><right><root>`

Tree Traversals

✓ Depth-First

1- Preorder (DLR)

<root><left><right>



15 10 8 12 20 17 25

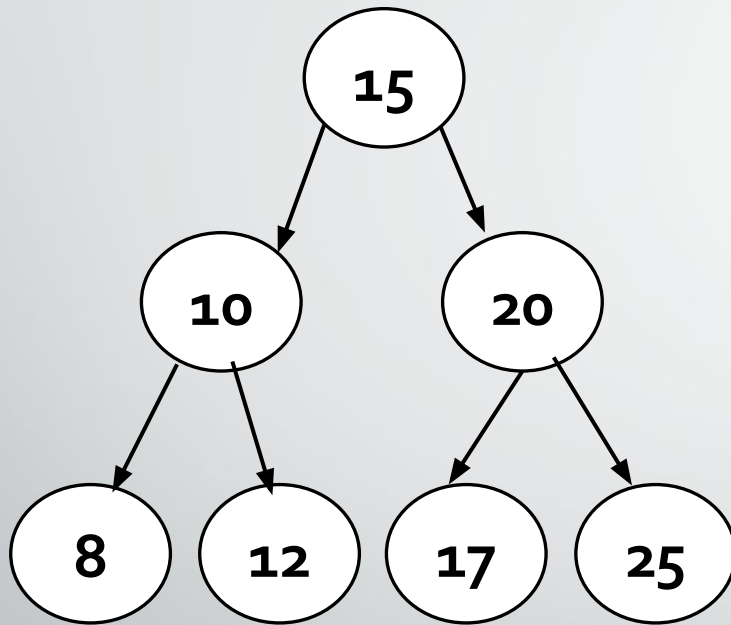
```
// Display using Depth-First (Preorder)
void displayDLR(Node *&root){
    if(root == NULL){
        return;
    }
    printf("%d ", root->data);
    displayDLR(root->left);
    displayDLR(root->right);
}
```

Tree Traversals

✓ **Depth-First**

2- Inorder (LDR)

<left><root><right>



8 10 12 15 17 20 25

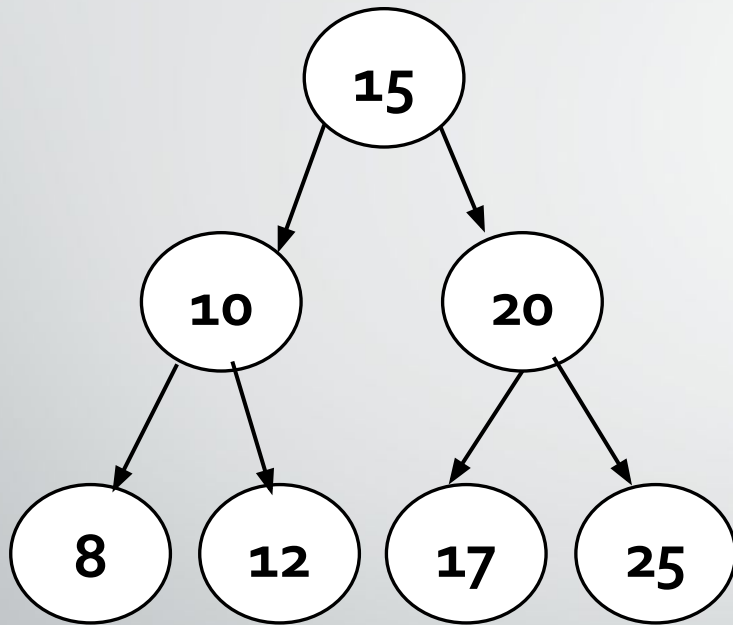
```
// Display using Depth-First (Inorder)
void displayLDR(Node *&root){
    if(root == NULL){
        return;
    }
    displayLDR(root->left);
    printf("%d ", root->data);
    displayLDR(root->right);
}
```

Tree Traversals

✓ **Depth-First**

3- Postorder (LRD)

<left><right><root>

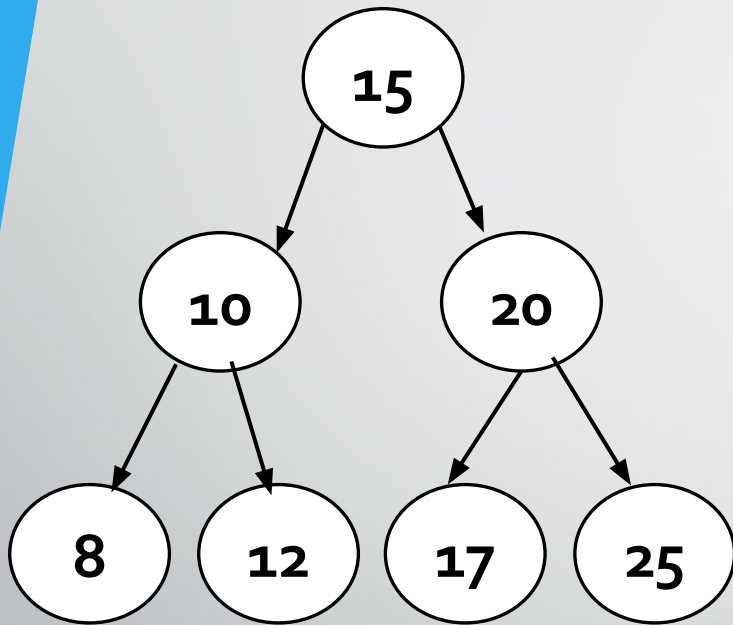


8 12 10 17 25 20 15

```
// Display using Depth-First (Postorder)
void displayLRD(Node *&root){
    if(root == NULL){
        return;
    }
    displayLRD(root->left);
    displayLRD(root->right);
    printf("%d ", root->data);
}
```

Print using Breadth-First

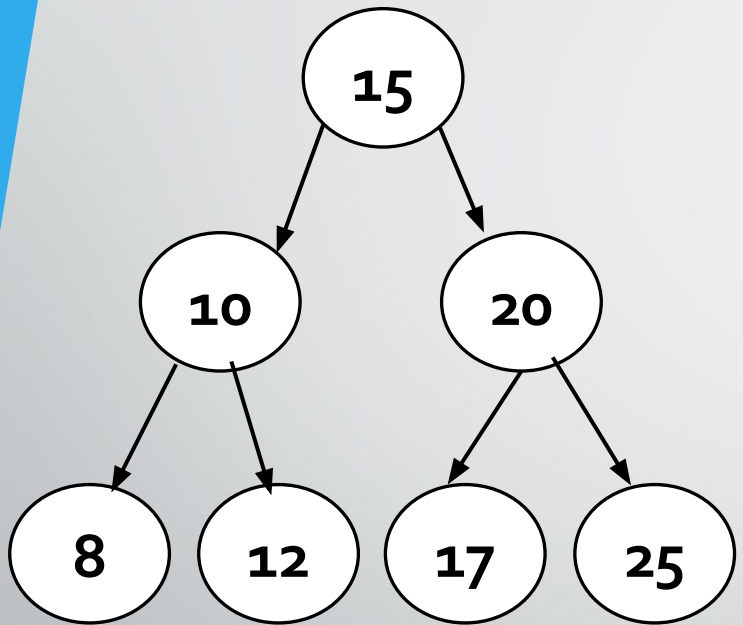
✓ Check if tree is empty



```
void display(Node *&root){  
    if(root == NULL){  
        printf("Empty BST");  
        return;  
    }  
}
```

Print using Breadth-First

✓ Create queue to save links



```
void display(Node *&root){  
    if(root == NULL){  
        printf("Empty BST");  
        return;  
    }  
    initialize(q);
```

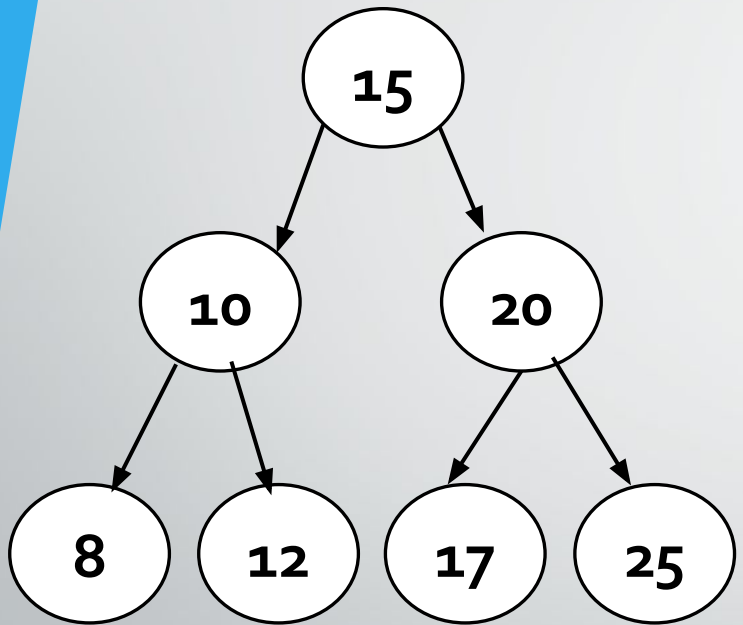
head

tail

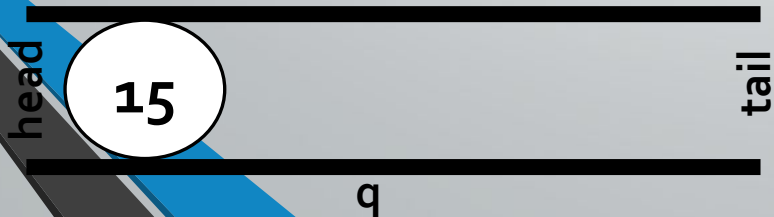
q

Print using Breadth-First

✓ Add the root to the queue

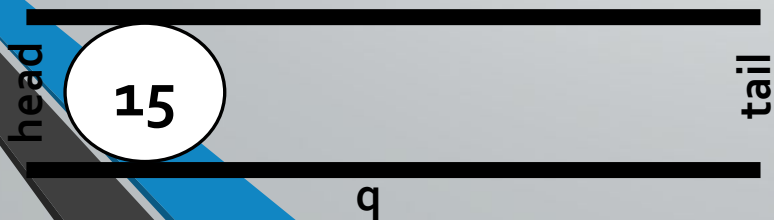
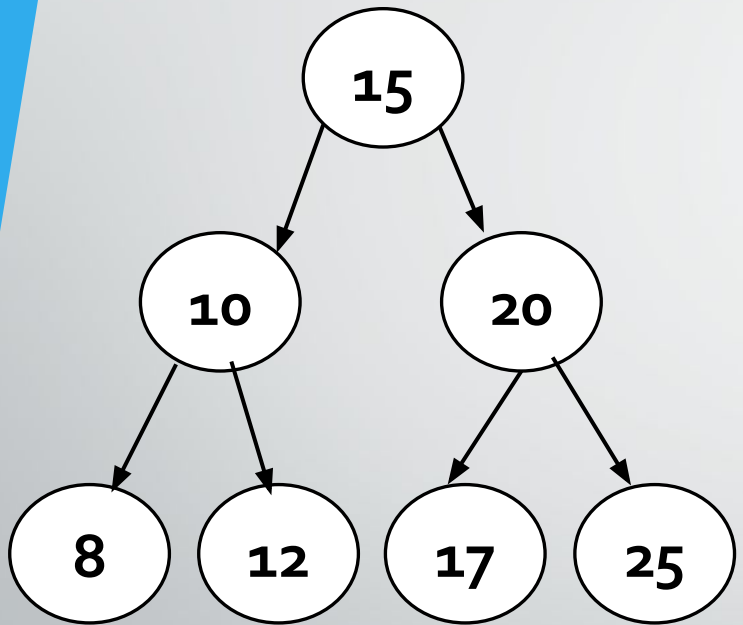


```
void display(Node *&root) {  
    if(root == NULL) {  
        printf("Empty BST");  
        return;  
    }  
    initialize(q);  
    Add(q, root);  
}
```



Print using Breadth-First

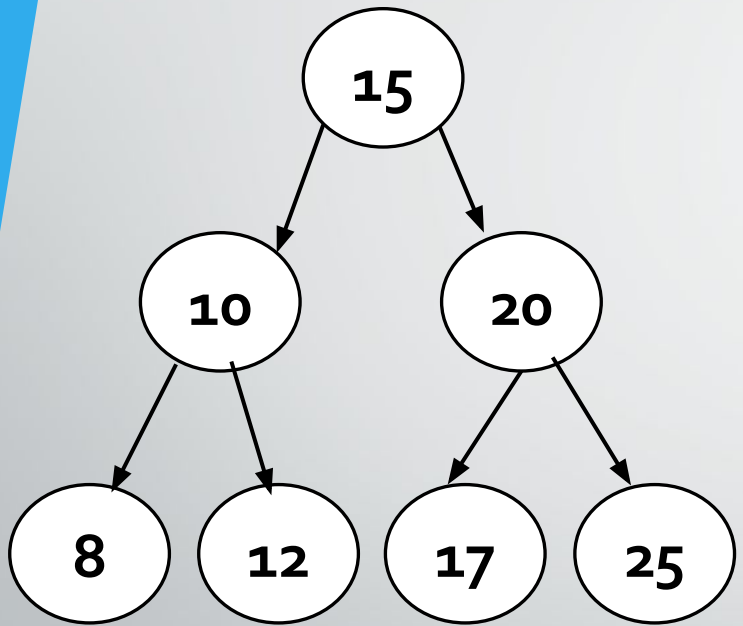
- ✓ Print the first element of the queue then If there's a children add it to the queue , ~~then remove the first element~~



```
void display(Node *&root) {
    if(root == NULL) {
        printf("Empty BST");
        return;
    }
    initialize(q);
    Add(q, root);
    Remove(q, current, empty);
    while(!empty) {
        printf("%d ", current->data);
        if(current->left!=NULL) {
            Add(q, current->left);
        }
        if(current->right!=NULL) {
            Add(q, current->right);
        }
        Remove(q, current, empty);
    }
}
```

Print using Breadth-First

✓ Output: 15



```
void display(Node *&root) {
    if(root == NULL) {
        printf("Empty BST");
        return;
    }
    initialize(q);
    Add(q, root);
    Remove(q, current, empty);
    while(!empty) {
        printf("%d ", current->data);
        if(current->left != NULL) {
            Add(q, current->left);
        }
        if(current->right != NULL) {
            Add(q, current->right);
        }
        Remove(q, current, empty);
    }
}
```

15

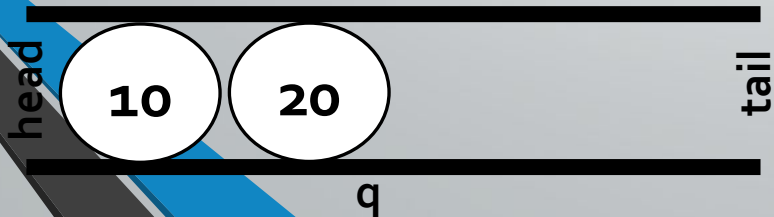
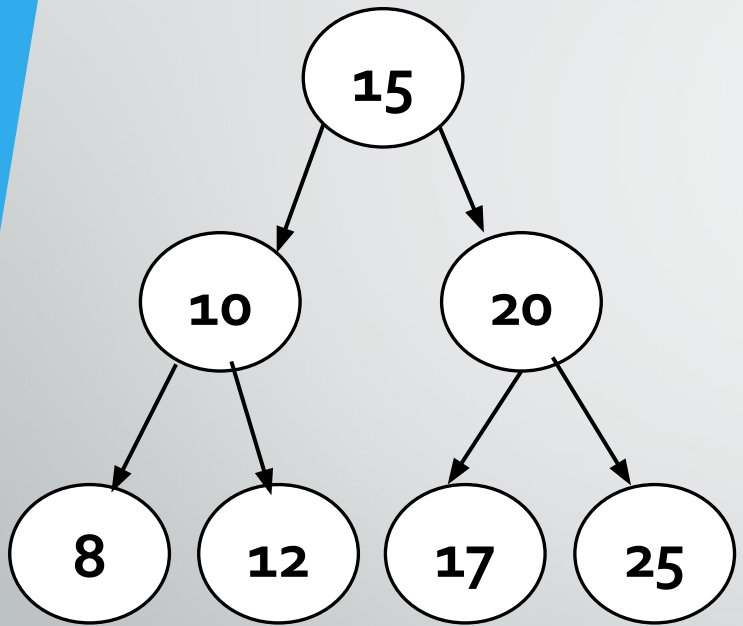
head

tail

q

Print using Breadth-First

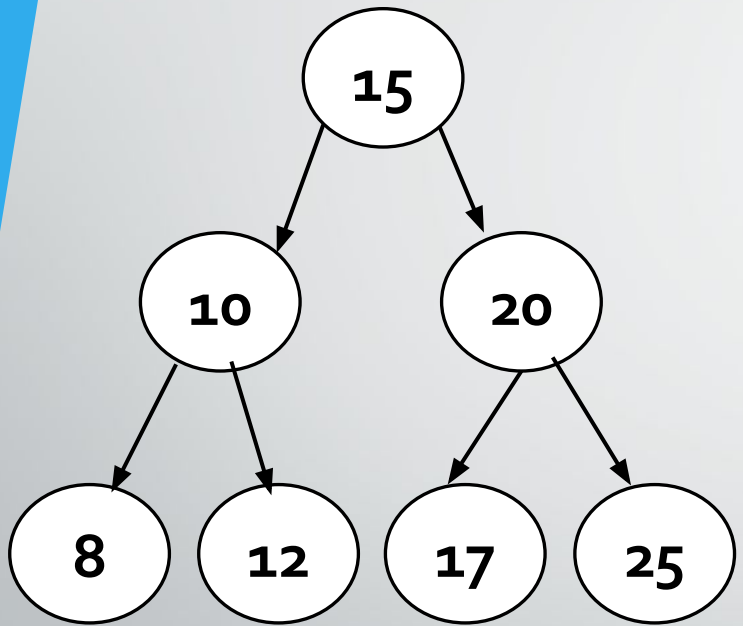
✓ Output: 15



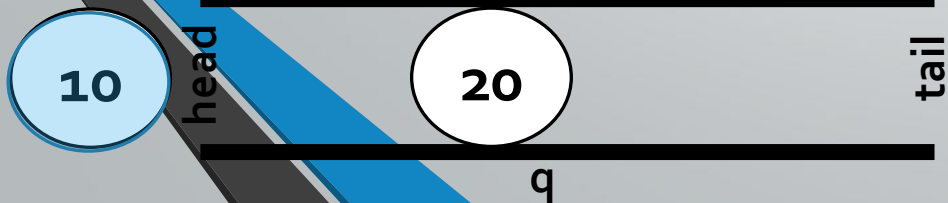
```
void display(Node *&root) {
    if(root == NULL) {
        printf("Empty BST");
        return;
    }
    initialize(q);
    Add(q, root);
    Remove(q, current, empty);
    while(!empty) {
        printf("%d ", current->data);
        if(current->left!=NULL) {
            Add(q, current->left);
        }
        if(current->right!=NULL) {
            Add(q, current->right);
        }
        Remove(q, current, empty);
    }
}
```

Print using Breadth-First

✓ Output: 15 10

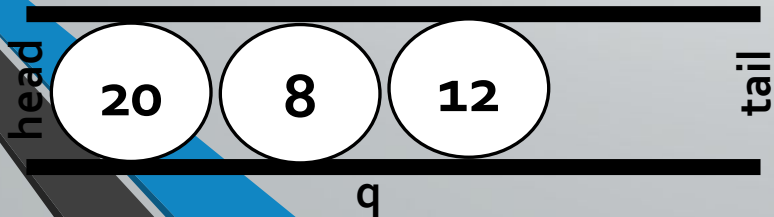
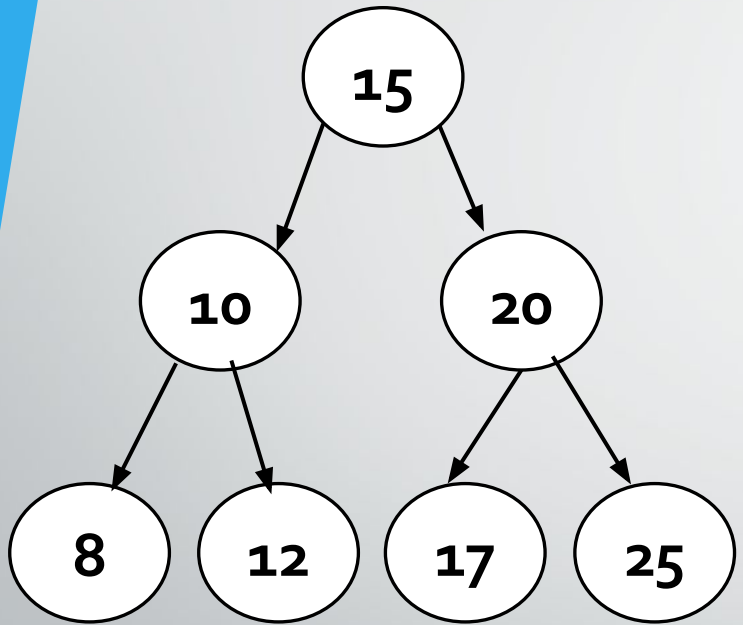


```
void display(Node *&root) {
    if(root == NULL) {
        printf("Empty BST");
        return;
    }
    initialize(q);
    Add(q, root);
    Remove(q, current, empty);
    while(!empty) {
        printf("%d ", current->data);
        if(current->left != NULL) {
            Add(q, current->left);
        }
        if(current->right != NULL) {
            Add(q, current->right);
        }
        Remove(q, current, empty);
    }
}
```



Print using Breadth-First

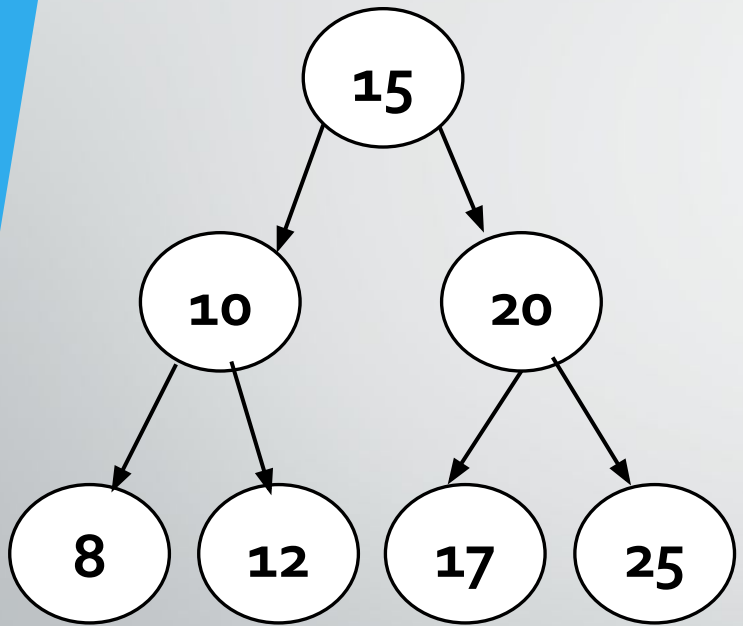
✓ Output: 15 10



```
void display(Node *&root) {
    if(root == NULL) {
        printf("Empty BST");
        return;
    }
    initialize(q);
    Add(q, root);
    Remove(q, current, empty);
    while(!empty) {
        printf("%d ", current->data);
        if(current->left != NULL) {
            Add(q, current->left);
        }
        if(current->right != NULL) {
            Add(q, current->right);
        }
        Remove(q, current, empty);
    }
}
```

Print using Breadth-First

✓ Output: 15 10 20

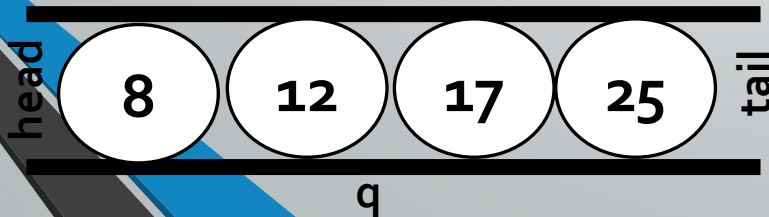
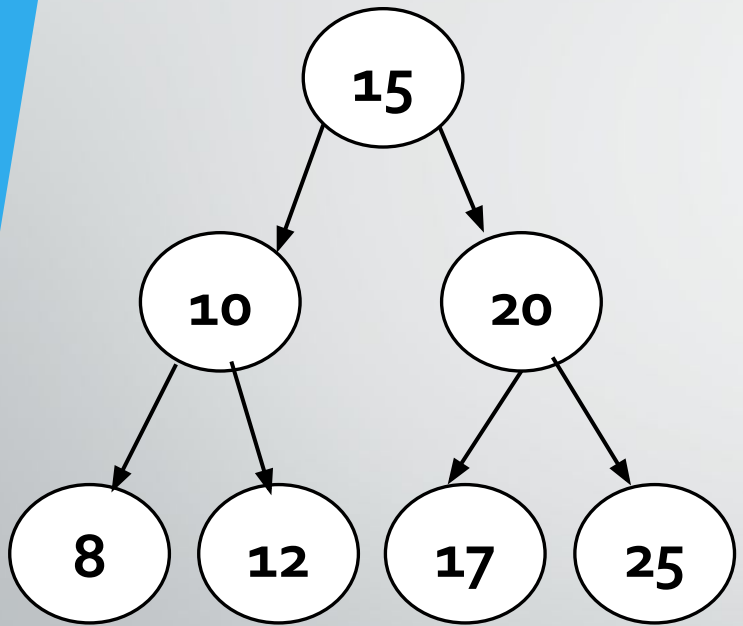


```
void display(Node *&root) {
    if(root == NULL) {
        printf("Empty BST");
        return;
    }
    initialize(q);
    Add(q, root);
    Remove(q, current, empty);
    while(!empty) {
        printf("%d ", current->data);
        if(current->left != NULL) {
            Add(q, current->left);
        }
        if(current->right != NULL) {
            Add(q, current->right);
        }
        Remove(q, current, empty);
    }
}
```



Print using Breadth-First

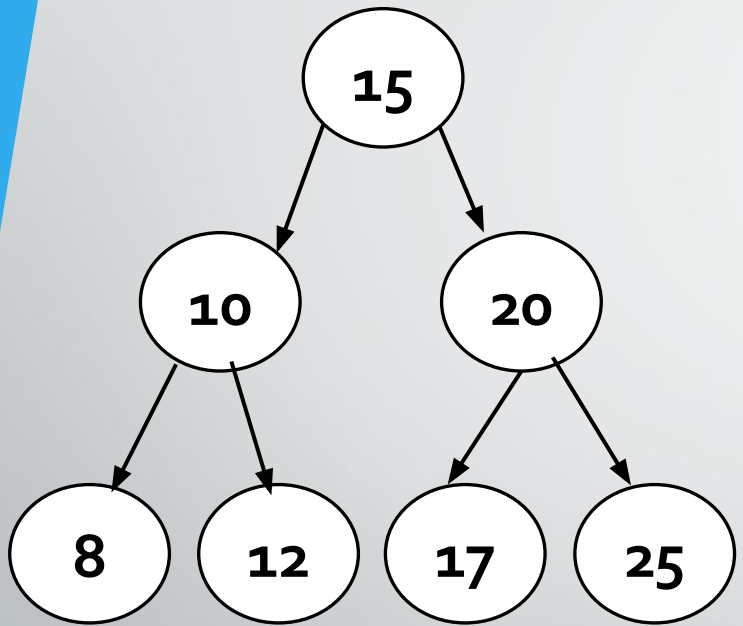
✓ Output: 15 10 20



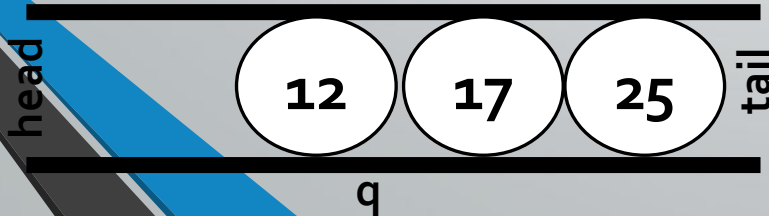
```
void display(Node *&root) {
    if(root == NULL) {
        printf("Empty BST");
        return;
    }
    initialize(q);
    Add(q, root);
    Remove(q, current, empty);
    while(!empty) {
        printf("%d ", current->data);
        if(current->left != NULL) {
            Add(q, current->left);
        }
        if(current->right != NULL) {
            Add(q, current->right);
        }
        Remove(q, current, empty);
    }
}
```

Print using Breadth-First

✓ Output: 15 10 20 8

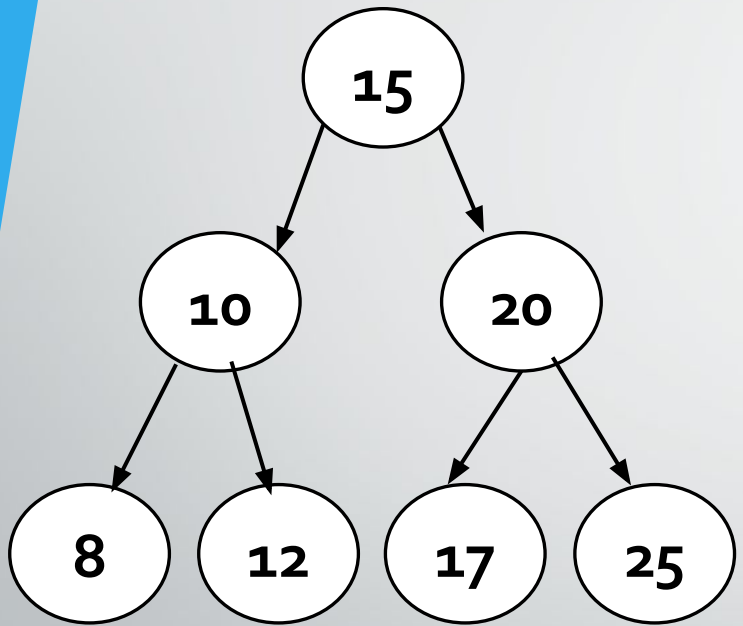


```
void display(Node *&root) {  
    if (root == NULL) {  
        printf("Empty BST");  
        return;  
    }  
    initialize(q);  
    Add(q, root);  
    Remove(q, current, empty);  
    while (!empty) {  
        printf("%d ", current->data);  
        if (current->left != NULL) {  
            Add(q, current->left);  
        }  
        if (current->right != NULL) {  
            Add(q, current->right);  
        }  
        Remove(q, current, empty);  
    }  
}
```



Print using Breadth-First

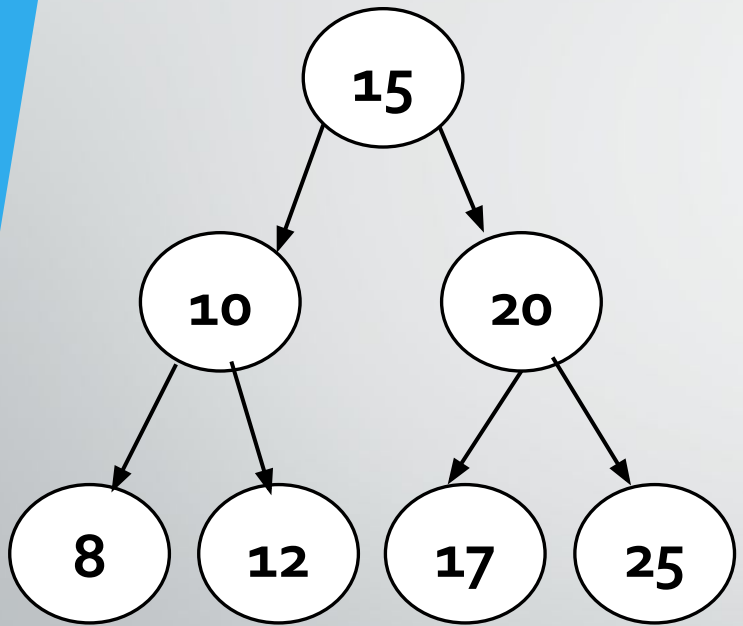
✓ Output: 15 10 20 8 12



```
void display(Node *&root) {  
    if(root == NULL) {  
        printf("Empty BST");  
        return;  
    }  
    initialize(q);  
    Add(q, root);  
    Remove(q, current, empty);  
    while(!empty) {  
        printf("%d ", current->data);  
        if(current->left != NULL) {  
            Add(q, current->left);  
        }  
        if(current->right != NULL) {  
            Add(q, current->right);  
        }  
        Remove(q, current, empty);  
    }  
}
```

Print using Breadth-First

✓ Output: 15 10 20 8 12 17



```
void display(Node *&root) {  
    if(root == NULL) {  
        printf("Empty BST");  
        return;  
    }  
    initialize(q);  
    Add(q, root);  
    Remove(q, current, empty);  
    while(!empty) {  
        printf("%d ", current->data);  
        if(current->left != NULL) {  
            Add(q, current->left);  
        }  
        if(current->right != NULL) {  
            Add(q, current->right);  
        }  
        Remove(q, current, empty);  
    }  
}
```

17

head

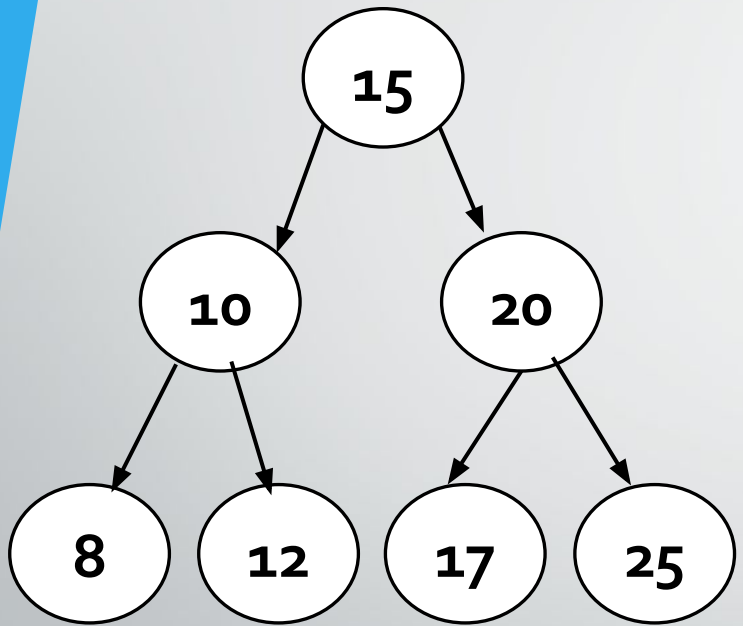
25

tail

q

Print using Breadth-First

✓ Output: 15 10 20 8 12 17 25

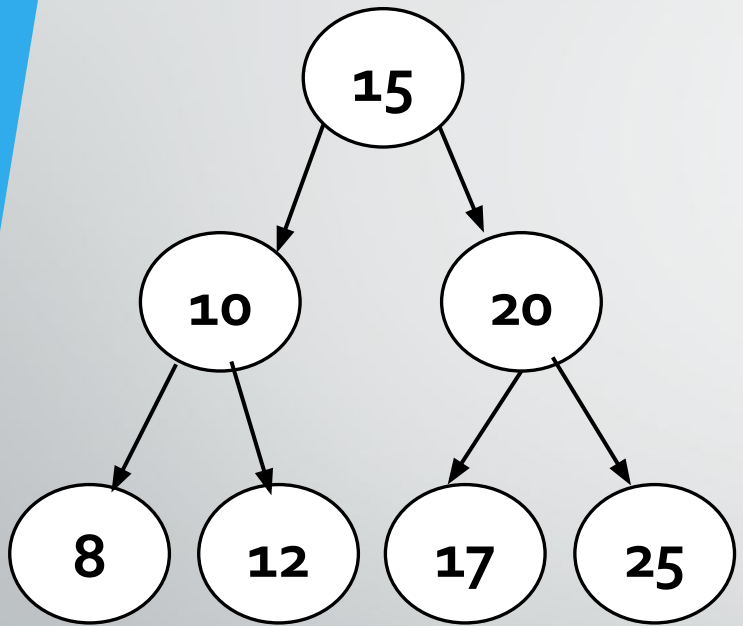


```
void display(Node *&root) {
    if(root == NULL) {
        printf("Empty BST");
        return;
    }
    initialize(q);
    Add(q, root);
    Remove(q, current, empty);
    while(!empty) {
        printf("%d ", current->data);
        if(current->left != NULL) {
            Add(q, current->left);
        }
        if(current->right != NULL) {
            Add(q, current->right);
        }
        Remove(q, current, empty);
    }
}
```



Print using Breadth-First

✓ Output: 15 10 20 8 12 17 25



```
void display(Node *&root) {
    if(root == NULL) {
        printf("Empty BST");
        return;
    }
    initialize(q);
    Add(q, root);
    Remove(q, current, empty);
    while(!empty) {
        printf("%d ", current->data);
        if(current->left != NULL) {
            Add(q, current->left);
        }
        if(current->right != NULL) {
            Add(q, current->right);
        }
        Remove(q, current, empty);
    }
}
```

END of Section

Thank You 